UNDERSTAND. DRAFT. SIMULATE.

Policy Studio.

Read what a policy actually does, draft a narrow rule, and see what it would change before you save anything.

Policies

Workspace policy packs/starter.yaml	Simulate
fetch / HTTP MCP servers (SSRF defense) built-in pack · fetch	Simulate
MCP filesystem server built-in pack · filesystem	Simulate
git MCP servers (run_git_command style tools) built-in pack · git	Simulate
Postgres / SQL MCP servers built-in pack · postgres	Simulate
shell / exec MCP servers built-in pack · shell	Simulate
Slack / messaging MCP servers built-in pack · slack	Simulate
starter baseline built-in pack · starter	Simulate

Explanations come from the same engine that decides your calls.

2	1	0	0
Rules	Command-flow rules	Broad permissions	Findings

e matches: Block. The call is refused and the request is not forwarded to the server. Denial alone does not prove a tool never ran, because a denied call may still have a recorded result.

No structural findings: no broad permissions, no unreachable rules, no conditions that cannot match. This describes the rules, not the safety of your agent.

Rules and precedence

#1 Allow **allow-reads**

Allow calls to tools matching "read_*", "get_*", "list_*", "search_*" and the tool "fetch".

Recorded explanation: read-only tools are routine

Tools

calls to tools matching "read_*", "get_*", "list_*", "search_*" and the tool "fetch"

Scope

any task label; no expiry; no argument conditions

What the boundary does

The call proceeds without asking anyone. The boundary still applies limits, result scanning, redaction and schema checks.

#2 Require approval **confirm-writes**

Require human approval for calls to tools matching "write_*", "edit_*", "create_*", "delete_*", "send_*", "post_*", "update_*", "exec*".

Recorded explanation: state changes need a human in the loop

Tools

calls to tools matching "write_*", "edit_*", "create_*", "delete_*", "send_*", "post_*", "update_*", "exec*"

Scope

any task label; no expiry; no argument conditions

What the boundary does

The call is parked until a human chooses allow once, allow for the session, or deny. With no terminal and no --auto-confirm the call is denied.

Command-flow rules

F1 Block **no-exfil**

Blocks calls to tools matching "send_*", "post_*", "exec*" once a permitted source call has run (source tools ching "read_*", "query_*").

Recorded explanation: data read inside the boundary must not leave it

Source tools

read_*, query_*

Sink tools

send_*, post_*, exec*

Mode

session taint (conservative)

Session limits

· at most 300 tool-call attempts per session

· at most 60 attempts in any sliding minute

· tool results larger than 2097152 bytes are replaced with an error

Limits are enforced by the boundary around each call, not by rule evaluation, so they do not appear as a rule decision. Attempts denied by policy still count.

Result injection scanning: log. Results are scanned and matches are recorded, but the call is not blocked.

▶ Original policy source

GUIDED BUILDER

Draft one rule.

The builder writes only rules the engine supports. Anything more complex stays in the source view, where it is never rewritten.

From recorded call #9 · packages__lookup · recorded decision: allow

This proposal comes from a single recorded call. One call shows what happened once, not what the agent intended or will do next.

A rule matches the tool name, so it governs every future call to packages__lookup — not only calls that look like this one.

Nothing here is applied. Review the rule, simulate it, and save it yourself.

This call also has a recorded result. Adding a deny rule does not remove recorded evidence.

ment names you may match on

What should happen to these calls?

Require human approval

The call is parked until a human chooses allow once, allow for the session, or deny. With no terminal and no auto-confirm the call is denied.

Which tools? (comma separated, exact names are safest)

packages__lookup

A * wildcard is supported by the engine but widens the rule. The preview flags broad patterns.

Why? (recorded with every decision)

package lookups of sensitive packages need a hu

Rule id (optional)

generated from the tool name

Where should this rule go?

New candidate file

A new candidate contains only this rule. Extending a policy copies it and inserts the rule first; nothing on disk changes.

Default for a new candidate

Allow anything not matched (allow)

Simulate against recording

Workspace audit · 1 deny

Simulation re-evaluates that recording. It never calls a tool.

▶ Advanced: argument conditions, task scope and expiry

Preview candidate →

Clear draft

Previewing validates the rule with the policy compiler and, when a recording is selected, simulates it against the calls you already recorded. No tool is executed and no file is written.

T ACTIVE

Review before you save.

(generated)

Rule id

New document

Placement

1

Rules in candidate

Compiles

Compiler

Validated as a standalone candidate. It does not inherit rules from any other policy.

This is a new policy document with default: allow. It contains only the rule you built; it does not inherit any rule from another policy.

With default deny, every call this rule does not match is blocked.

The policy default is allow: any call that matches no rule proceeds. Only a rule that matches stops it.

This candidate would permit 1 call(s) that the recording shows as denied.

This text is a proposal. The workbench has not written it anywhere and nothing will use it until you point `tapelog record --policy` at the file you save.

```
# Tapelog policy candidate — generated by the Session Workbench Policy Studio.
#
# This is a proposal, not an active configuration. Tapelog never activates a
# candidate for you: review it, simulate it, then save it yourself.
#
# Simulate it against a recording with:
#   tapelog policy whatif --policy <this file> <recording.jsonl>
# Try sample calls with:
#   tapelog policy test --policy <this file> --calls calls.jsonl

version: 1
default: allow
rules:
  - id: confirm-packages-lookup
    tool:
      - packages__lookup
    action: confirm
    reason: package lookups of sensitive packages need a human
```

as... Copy YAML Discard

sha256 d222cb95d013940ba41e76a5767a00a7739dc78fedf51e5cec57a61e1f7e1803

▶ Check it yourself

VISUAL WHAT-IF

What would change?

Against this recording the candidate changes 2 of 4 recorded decisions.

4	2	0	1	1
Calls re-evaluated	Decisions changed	Newly blocked	Newly allowed	New approvals

Changes only (2) | Newly blocked (0) | Newly allowed (1) | New approvals (1) | All calls (4)

#3 **packages__lookup** Allow → Require approval

Recorded

Allow · rule read-only-scope

Candidate

Require approval · rule confirm-packages-lookup

package lookups of sensitive packages need a human

#4 **workspace__delete_file** Block → Allow

Recorded

Block · rule protect-project-files

Candidate

Allow · rule default

no rule matched; policy default is "allow"

This call was recorded without a result.

The policy default is allow: any call that matches no rule proceeds. Only a rule that matches stops it.

vould permit 1 call(s) that the recording shows as denied.

Re-evaluated with the boundary's own evaluation pipeline: value-level taint rebuilt from recorded results, then session flows, then ordered rules. Recorded (redacted) arguments are used, so verdicts reproduce exactly. No tool call is executed.

This shows what the candidate would have decided about activity that was already recorded. It does not prove how future, unseen calls will behave, and it cannot see traffic that bypassed this boundary.

- MCP-mediated activity only. Built-in agent operations are outside Tapelog’s view.